How Attack Data Is Built Decides Which Attacks Look Detectable: Counterexamples from Agent Monitoring

Bipin Rimal
Independent Researcher
bipinrimal314@gmail.com

***Abstract*—Behavioural anomaly detectors for AI agents are often evaluated in two ways: by injecting synthetic attack profiles into agent traces, and by computing AUC-ROC on public trace benchmarks. We re-examine our own earlier evaluation and find that the construction of the data decided each answer. First, two constructions of the same four OWASP agentic risk categories disagree. With negatives held out of training, tool misuse (ASI02) scores 0.50–0.51 under our perturbation profiles and 0.82–0.89 on ATBench, an independently constructed benchmark. Memory and context poisoning (ASI06) errs the other way (0.89–0.96 against 0.79–0.89). Second, neither side measures attack behaviour. Our ASI02 profile changes no span, tool or order, so it is undetectable by construction. On ATBench, the raw number of tool calls in a trajectory, with no model at all, scores 0.89–0.94, higher than all three detectors in all six categories. Third, TRAIL has four error-free traces among 148. A detector that scored 0.89 when its four training negatives were also its test negatives scores 0.60 when each is held out, and the 95% interval on a TRAIL AUC-ROC is about $\pm$0.25. We withdraw two earlier claims. We recommend held-out negatives, a trace-length baseline, an interval on every AUC, and a second construction before any category is called a blind spot.**

I. Introduction

Monitoring AI agents with behavioural anomaly detection is attractive. It needs no labels at training time, and it reuses decades of work on insider-threat detection. Evaluating such a monitor is harder than building it. Labelled agent attacks are rare, so two shortcuts are common. The first shortcut injects synthetic attack profiles into existing traces and measures how well the detector finds them, one risk category at a time. The second shortcut reports AUC-ROC on a public trace benchmark.

This paper reports what happened when we checked both shortcuts in our own earlier work [1]. That work used synthetic profiles to name tool misuse (OWASP ASI02 [2]) as a structural blind spot. It also reported that a detector transferred from another domain scored higher on TRAIL [3] than one trained on TRAIL itself. Neither claim survives.

We make three points.

1) **Construction decides detectability.** Two constructions of the same OWASP categories disagree by up to 0.39 AUC-ROC per category, in both directions (Section III).
2) **No detector improves on a count.** On ATBench, the number of tool calls alone separates unsafe from safe trajectories at least as well as any detector we trained (Section IV).
3) **TRAIL cannot rank detectors.** With four negatives, a score depends on whether those negatives were in the training set, and the 95% interval spans about $\pm$0.25 (Section V).

These are counterexamples, not a general law. Each one is small and specific. We think their value is that each is cheap to check in other evaluations.

II. Setup

Features. Each trace becomes a 20-dimensional Unified Behavioural Feature Schema (UBFS) vector in seven groups: temporal, frequency, volume, scope, sequence, deviation and privilege [1]. The features describe behaviour only: timing, counts, tool breadth, action entropy, transition novelty and similar. No feature reads the text of a prompt, a tool argument or an output.

Detectors. We use Isolation Forest (IF) [4] with 200 trees, an LSTM autoencoder (LSTM) [5], and a deep-clustering detector (DC), a dense autoencoder with k-means in its latent space [6]. Each is trained on normal traces only. Every result is a mean over five seeds.

Protocol. Our earlier work trained each detector on all normal traces and then scored those same traces as the negatives. Here the test negatives are never in the training set. For the synthetic data and for ATBench we split the normal traces into five folds, train on four, and test on the fifth fold plus all positives of one category. For TRAIL we leave one error-free trace out at a time. We also report the earlier protocol as "in-sample". One script, `scripts/heldout_check.py`, produces every number in Tables I, II and III.

Count baseline. As a baseline with no model, we score each trace by its raw number of tool calls. If fewer calls marked the positive class we would reverse the sign. No cell in this paper needed that.

Synthetic construction. We inject attack profiles into TRAIL traces. Each profile is a rule that edits a base trace to match our idea of one OWASP category. For example, the goal-hijack profile (ASI01) inserts unfamiliar tool calls mid-trace, and the memory-poisoning profile (ASI06) duplicates the session's spans several times. The code is in `src/data/synthetic_generator.py`. The base set

TABLE I
PER-CATEGORY AUC-ROC WITH HELD-OUT NEGATIVES (UBFS-20, MEAN OF FIVE SEEDS AND FIVE FOLDS). SYN.: OUR PERTURBATION PROFILES ON TRAIL (29–30 POSITIVES, 148 NORMAL). ATB.: ATBENCH (29–73 POSITIVES, 250 SAFE). COUNT: RAW NUMBER OF TOOL CALLS, NO MODEL.

	IF		LSTM		DC		Count	
Cat.	Syn.	ATB.	Syn.	ATB.	Syn.	ATB.	Syn.	ATB.
ASI01	.610	.842	.616	.780	.595	.872	.635	**.919**
ASI02	.503	.870	.503	.819	.506	.894	.531	**.941**
ASI06	.894	.866	.959	.794	.964	.886	.784	**.939**
ASI10	.690	.763	.673	.707	.713	.810	.763	**.889**

is all 148 TRAIL traces, and the unedited traces serve as the normal class. They are not clean: 144 of them contain annotated agent errors.

Independent construction. ATBench [7] provides 1,000 multi-turn tool-calling trajectories with safety labels and fine-grained risk categories. We use its 500-trajectory configuration (250 safe, 250 unsafe) and map its risk categories to OWASP ASI codes. ATBench is not a record of real incidents. Its authors construct the trajectories and then filter them with rules, language models and a human audit. We use it as a second construction of the same categories, built by people who did not see our profiles.

Trace benchmark. TRAIL [3] contains 148 agent traces from GAIA and SWE-bench tasks with annotated errors. In 144 of them at least one error is annotated, so four traces are negatives. TRAIL was designed for error localisation, not for trace-level detection, so our use of it is outside its design intent.

III. CONSTRUCTION DECIDES DETECTABILITY

Table I compares the four categories that both constructions share. The detectors and features are identical. The data changes on both sides: the attacks, and also the normal class (TRAIL traces against ATBench safe trajectories). We cannot separate the two, and we return to this in Section VIII.

The two constructions disagree in both directions. Under our profiles, tool misuse (ASI02) sits at chance for all three detectors (0.50–0.51). On ATBench the same detectors score 0.82–0.89 on the same category, a gap of 0.32–0.39. Memory and context poisoning (ASI06) errs the other way: our profiles make it look easier (0.89–0.96) than ATBench does (0.79–0.89). The ASI02 gap is larger than the sampling error. With the number of negatives in one test fold, the Hanley–McNeil [8] 95% interval is 0.35–0.65 for each synthetic ASI02 score, and its lower bound is 0.71 or more for each ATBench ASI02 score.

The ASI02 gap follows from the generator. Our ASI02 profile does not add, remove or reorder any span, and it does not change any tool name. It only rescales existing numeric span attributes, such as token counts, by a small random factor. The count, scope and sequence features therefore do not change. A behaviour-only detector has almost nothing to see, whatever real tool misuse looks like. The chance-level score measures our definition of the category, not the category. Under the earlier in-sample protocol, adding eight semantic features to the schema left synthetic ASI02 near chance (DC: 0.537 to 0.559; IF: 0.514 to 0.494).

What the comparison does not show. ATBench is also a construction. The disagreement shows that the answer depends on who builds the data. It does not show which construction is closer to attacks in deployment. Our earlier write-up reported a perfect rank inversion over three shared categories. With the category mapping corrected, four categories are shared, and no rank correlation over four items can reach significance (the smallest attainable exact two-sided p is 0.083), so we do not use one. The evidence is the size of the per-category gaps and the known mechanism.

IV. NO DETECTOR IMPROVES ON A COUNT

The last two columns of Table I give the count baseline. On ATBench it scores 0.89–0.94 and is higher than every detector in every category, including the two categories in Table III that have no synthetic profile. The margin over the best detector is 0.05–0.08, which is about the half-width of the intervals, so we do not claim that the count is reliably better. The supported claim is that no detector improves on it.

The same cue is visible in the raw data, before any of our code runs. There it appears as the message count, which is the same quantity as our tool-call count up to scale. A safe trajectory in the 500-trajectory configuration has a median of 8 messages, and an unsafe one has a median of 10. The message count alone separates the two classes with an AUC-ROC of 0.924. Of the 250 unsafe trajectories, 226 have ten or more messages, and of the 250 safe ones, 213 have eight or fewer. The number of tools offered to the agent does not separate them (0.509).

A single feature therefore does as well on ATBench as a trained detector. We did not remove the count features and retrain, so we cannot say how much of each detector's score comes from length. The ATBench column of Table I does not show that tool misuse is detectable from behaviour. It is consistent with a simpler reading: unsafe trajectories in this construction are longer. Both constructions decided the answer: ours by changing nothing a feature can read, and ATBench by a difference in length. We did not test the 1,000-trajectory configuration, and we make no claim about why the lengths differ.

Under our own profiles the count baseline is also competitive. It is above all three detectors for ASI01 and ASI10, and below them only for ASI06, where the profile duplicates spans.

V. TRAIL CANNOT RANK DETECTORS

AUC-ROC is the probability that a random positive scores above a random negative [8]. With four negatives, every AUC on TRAIL is an average over the ranks of four traces.

The training set decided the ranking. Under the in-sample protocol, DC scores 0.893 and looks far better than IF (0.577). When each negative is held out, DC scores 0.598 and is the lowest of the three (Table II). A likely cause is that a model

TABLE II
AUC-ROC ON TRAIL (144 POSITIVES, 4 NEGATIVES; MEAN OF FIVE SEEDS). IN-SAMPLE: THE FOUR NEGATIVES ARE THE TRAINING SET. HELD-OUT: EACH NEGATIVE IS LEFT OUT OF TRAINING IN TURN. INTERVAL: APPROXIMATE HANLEY–MCNEIL 95% INTERVAL AROUND THE HELD-OUT SCORE.

Detector	In-sample	Held-out	Interval
IF	0.577	0.654	0.41–0.90
LSTM	0.685	0.675	0.44–0.91
DC	0.893	0.598	0.34–0.86
Count (no model)	0.569		

fitted to four traces places those four traces close to its own cluster centres, so they receive low anomaly scores for a reason that has nothing to do with errors. We did not test this cause. IF and LSTM do not show the same drop. The earlier ranking was a property of the protocol.

No ordering is supported. The three held-out scores lie between 0.60 and 0.68, and each interval is about ± 0.25 wide. Our earlier work reported that an IF trained on TRACE [9] (0.614) or on ATBench (0.774) scored higher on TRAIL than an IF trained on TRAIL. Both transferred scores lie inside the interval around the held-out in-domain score (0.41–0.90). The pattern is not evidence of anything.

Seed variation hides the problem. The seed standard deviations we reported earlier were small (0.007–0.050). They measure the randomness of model training. They do not measure the sampling of the four negatives, which is the dominant source of error. A table of means and seed deviations therefore looks precise when it is not.

The cause is few negatives, not prevalence as such. AUC-ROC does not depend on class balance in principle [8], [10]. The problem is that four negative traces make the statistic too noisy to compare anything. AUC-PR does not help: IF's in-sample AUC-PR is 0.982, close to what any scorer achieves when 97% of traces are positive [11].

VI. ONE SYNTHETIC PREDICTION THAT HELD

Our earlier work made a second synthetic prediction of the same kind. It modelled distributed model distillation: extraction traffic spread over many accounts, so that each account looks normal. Under the in-sample protocol all three detectors stayed near chance on this profile (0.538–0.540), and a single-account extraction profile was easy to detect (0.936–0.999). A later public disclosure [12] described distillation campaigns spread across many accounts, and detection tools built for coordinated activity across accounts. This agrees with the prediction in kind. It is not a measurement, because no labelled benchmark of distributed extraction was available to us.

The ASI02 prediction and this one came from the same pipeline. Both were near-chance scores on profiles we defined, and both had a plausible story attached. One agreed with later evidence and one did not. Nothing inside the synthetic experiment separated them.

VII. RELATED WORK

Synthetic data in detection. Critiques of the DARPA intrusion-detection evaluations showed that simulated traffic gives optimistic and artefact-driven results [13], [14], and these pitfalls remain common in security ML [15]. In AI control, Li et al. find that monitors trained on prompt-elicited reward hacking fail on hacking that arises during training [16]. Loughridge et al. report a case where synthetic data did transfer: attack policies tuned in a simulation of the control setting stayed effective on SHADE-Arena [17]. Their simulation was fitted to the target setting. Our ASI02 profile was fitted to nothing, which may explain the difference.

Evaluation validity. Zavrak shows that random splits inflate AUROC by up to 26 points on agent tool-call detection, that metadata-only features plateau near 0.64 AUROC, and that content embeddings push AUROC above 0.89 [18]. A validity audit of agent-safety benchmarks shows that an always-positive baseline can beat several models [19]. Wu and Keogh show how flawed anomaly benchmarks create an illusion of progress [20]. We found no prior report of a per-category disagreement between injected profiles and an independent benchmark, of a length cue in ATBench, or of TRAIL's four negatives as a limit on detector ranking. Our search was not exhaustive.

Prior work by the author. This paper revises results from the author's earlier paper [1], which reported the synthetic ASI02 blind spot and the TRAIL transfer result. Both are withdrawn here.

VIII. LIMITATIONS

Behaviour-only features. UBFS reads no content. Content-aware detectors exceed 0.89 AUROC on related data [18], so our detectors are weak by current standards. We did not compare against a language-model monitor.

Normal class differs. The two constructions also use different normal data. A gap in Table I can come from the attacks, from the normal class, or from both.

Single constructions. Each side of Table I is one construction: one profile set and one benchmark configuration. Only four categories are shared, and ASI02 has only 29 positive trajectories in ATBench. Other generators or benchmarks may agree with each other.

Synthetic positives resemble training traces. Each synthetic positive is an edited copy of a base trace, and that base trace is usually in the training folds. This can lower the synthetic scores, most of all for ASI02, whose edit is nearly an identity. Folds grouped by base trace would remove the effect. We did not run them.

Splits and normalisation. Our folds are random, not task-disjoint, and Zavrak shows that this choice alone can inflate AUROC [18]. The z-score normaliser is fitted on all traces of a dataset, including the test traces.

Intervals. The Hanley–McNeil interval assumes one scorer. Our held-out scores average over folds with different models, so the intervals are approximate.

Reproduction. With the earlier in-sample protocol, this run reproduces our earlier IF and LSTM numbers to three decimals. The DC numbers on ATBench come out 0.02–0.04 lower than before (for example, ASI02 0.888 against 0.910). The run is deterministic in our environment (PyTorch 2.15, CPU), so we attribute the difference to software or hardware, and we report only this run. The distillation numbers in Section VI and the semantic-feature numbers in Section III come from the earlier in-sample run and were not repeated with held-out negatives.

Lost insider-threat pipeline. Our earlier work also transferred detectors from the CMU-CERT insider-threat data. The feature-loading code for that data was never committed and is lost, so those numbers cannot be rerun. We do not use them here.

IX. Conclusion

How the data was built decided which attacks looked detectable in our evaluation. Our own profile made tool misuse invisible, a length difference made it easy on ATBench, and four negatives made any ranking on TRAIL depend on the protocol. Four practices follow. Hold the test negatives out of training. Report a trace-length baseline next to every detector. Report an interval for every AUC, and count the negatives first. Check a synthetic per-category score against a construction built by someone else before calling a category a blind spot.

References

[1] B. Rimal, "From rogue employees to rogue agents: Repurposing insider threat detection for AI agent governance," SSRN Working Paper 6355658, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6355658, 2026.

[2] OWASP GenAI Security Project, "OWASP top 10 for agentic applications for 2026," https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/, December 2025.

[3] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, "TRAIL: Trace reasoning and agentic issue localization," *arXiv preprint arXiv:2505.08638*, 2025.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*. IEEE, 2008, pp. 413–422.

[5] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "Lstm-based encoder-decoder for multi-sensor anomaly detection," in *ICML 2016 Anomaly Detection Workshop*, 2016.

[6] S. Zhou, H. Xu, Z. Zheng, J. Chen, Z. Li, J. Bu, J. Wu, X. Wang, W. Zhu, and M. Ester, "A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–38, 2025.

[7] Y. Li, H. Luo, Y. Xie, Y. Fu, Z. Yang, S. Shao, Q. Ren, W. Qu, Y. Fu, Y. Yang, J. Shao, X. Hu, and D. Liu, "ATBench: A diverse and realistic agent trajectory benchmark for safety evaluation and diagnosis," *arXiv preprint arXiv:2604.02022*, 2026, data: https://huggingface.co/datasets/AI45Research/ATBench.

[8] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982.

[9] D. Deshpande, A. Kannappan, and R. Qian, "Benchmarking reward hack detection in code environments via contrastive analysis," *arXiv preprint arXiv:2601.20103*, 2026.

[10] M. B. A. McDermott *et al.*, "A closer look at AUROC and AUPRC under class imbalance," in *Advances in Neural Information Processing Systems*, 2024, arXiv:2401.06091.

[11] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[12] Anthropic, "Detecting and preventing distillation attacks," https://www.anthropic.com/news/detecting-and-preventing-distillation-attacks, February 2026.

[13] J. McHugh, "Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA intrusion detection system evaluations as performed by Lincoln Laboratory," *ACM Transactions on Information and System Security*, vol. 3, no. 4, pp. 262–294, 2000.

[14] M. V. Mahoney and P. K. Chan, "An analysis of the 1999 DARPA/Lincoln Laboratory evaluation data for network anomaly detection," in *Recent Advances in Intrusion Detection (RAID)*, ser. LNCS, vol. 2820, 2003, pp. 220–237.

[15] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *USENIX Security Symposium*, 2022.

[16] L. Li, H. Zhou, Y. Liang, T. Zhou, and C.-J. Hsieh, "Do prompt-elicited trajectories reflect training-time reward hacking? A systematic study on monitoring training-time reward hacking in code generation," *arXiv preprint arXiv:2604.23488*, 2026.

[17] C. Loughridge, P. Colognese, A. Griffin, T. Tracy, J. Kutasov, and J. Benton, "Optimizing AI agent attacks with synthetic data," *arXiv preprint arXiv:2511.02823*, 2025.

[18] S. Zavrak, "Content-aware attack detection in LLM agent tool-call traffic: An empirical study of features, architectures, and evaluation protocols," *arXiv preprint arXiv:2605.11053*, 2026.

[19] Y. Wang, X. Han, D. Shang, Y. Tang, and B. Liu, "Safety, or just capability? A validity audit of agent-safety benchmarks," *arXiv preprint arXiv:2607.28685*, 2026.

[20] R. Wu and E. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *arXiv preprint arXiv:2009.13807*, 2020.

Appendix

A. Full ATBench Results

Table III gives held-out per-category AUC-ROC on ATBench for all six mapped categories. ASI04 and ASI08 have no synthetic profile. Holding the negatives out changes an ATBench score by at most 0.009, and a synthetic score by at most 0.044.

TABLE III
ATBench per-category AUC-ROC with held-out negatives (UBFS-20). n: positive trajectories.

	ASI01	ASI02	ASI04	ASI06	ASI08	ASI10
n	61	29	29	73	29	29
IF	.842	.870	.845	.866	.775	.763
LSTM	.780	.819	.797	.794	.727	.707
DC	.872	.894	.878	.886	.824	.810
Count	**.919**	**.941**	**.937**	**.939**	**.903**	**.889**

B. Interval Method

For an AUC A estimated from n_p positives and n_n negatives, Hanley and McNeil [8] give

$$\mathrm{SE}^2 = \big[A(1-A) + (n_p-1)(Q_1 - A^2) + (n_n-1)(Q_2 - A^2)\big]/(n_p\, n_n),$$

with $Q_1 = A/(2-A)$ and $Q_2 = 2A^2/(1+A)$. The 95% interval is $A \pm 1.96\,\mathrm{SE}$. With $n_n = 4$, each negative takes part in a quarter of all positive–negative pairs, so the interval stays wide however many positives there are.